

SESSION 1 OF 3 — FROM DICOM TO A GENERALIZED RESEARCH DATASET

From Pixels to Patients

Practical Pipelines for Implementing AI in Radiation Oncology



Session QR code – Follow along!

Brian Mark Anderson, PhD

Radiation Medicine & Applied Sciences

AAPM Annual Meeting · July 19, 2026

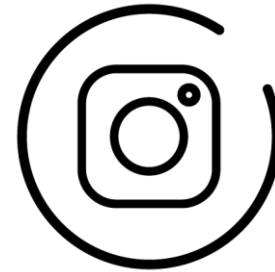
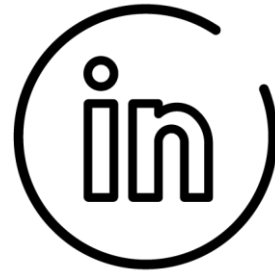
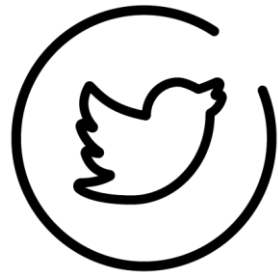
UC San Diego Health

RETHINKING MEDICAL PHYSICS



Note

Please feel free to photograph and share these slides on social media.





Disclosures

- Adjunct Assistant Professor, University of North Carolina, Chapel Hill
- Liminal-x



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

OVERVIEW

What you'll take away



01

Convert with confidence

Pre-process and convert DICOM to NIfTI for images, structures, and dose.

02

Keep what matters

Preserve the meta-features that drive research: age, voxel dimensions, and outcome scoring.

03

Grow the dataset

Maintain an anonymization key so you can keep collecting data and fold it back into the group.

TAKE IT WITH YOU

Open resources to run today



Reference notebook

End-to-end DICOM → NIfTI on the NSCLC-Radiomics cohort, manifest, outliers, resampling, export.



Public sample data

NSCLC-Radiomics from TCIA: download instructions included in the notebook.



Templates

Metadata sidecar and anonymization-key scaffolds to drop into your workflow.

github.com/brianmanderson/Dicom_RT_and_Images_to_Mask · `pip install DicomRTTool`

https://github.com/brianmanderson/AAPM2026_PixelsToPatients

THE SCENARIO

Our story for today



Let's act as if we've just pulled the NSCLC-Radiomics cohort from TCIA and are seeing it for the first time.

01

Preprocess and evaluate the data

Build a manifest of what we have, and screen it for outliers first.



02

Establish output size, chosen ROIs

Pick the voxel size we resample to, and the contours (ROIs) we keep.



03

Ensure meta-data comes along

Age, spacing, and outcomes travel with the images in a metadata sidecar.

Three steps, one cohort, start to finish — the same workflow you can rerun on your own data.

THE STARTING POINT

Sifting through the data



RAW INPUT



SIFT · PREPROCESS · Quality Control



ANALYSIS-READY

Garbage in → Garbage out

THE STARTING POINT

Sifting through the data: The Code

We want an efficient tool to **iterate down through our data**. Several exist; I recommend two I built.

PYTHON · DICOM → NIfTI

DicomRTTool Python

DicomRTGUI GUI

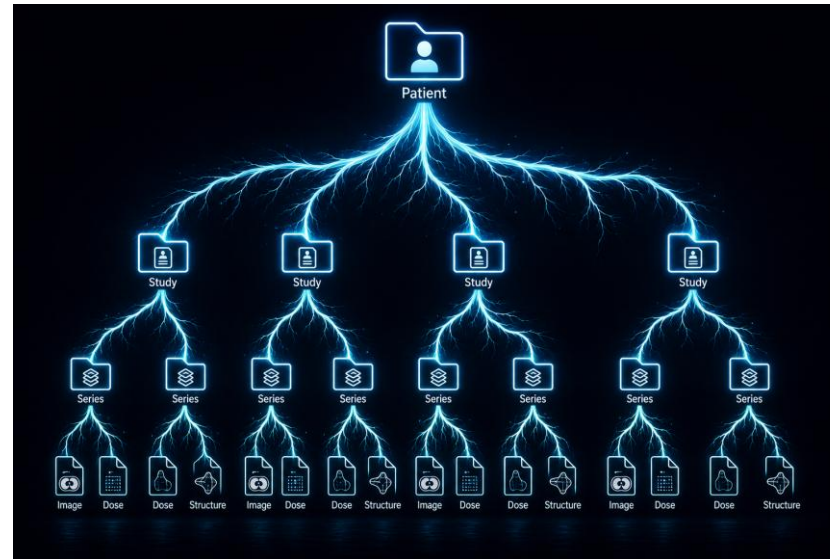
ALSO AVAILABLE

dcmrtstruct2nii · rt-utils · platipy ·
SlicerRT · dcm2niix

★ *The top two are mine — fair warning, I'm biased.*

Installation

```
pip install DicomRTTool
```



SCAN TO OPEN ON GITHUB



DicomRTTool
(Python)

pip install · GitHub

★ Recommended — grab the
Release



DicomRTGUI
(stand-alone GUI)



THE STARTING POINT

Sifting through the data: The Results

Installation

```
pip install DicomRTTool
```



Step 1 — Discover: walk the folders

`walk_through_folders` recursively scans a directory tree, groups files by `SeriesInstanceUID`, and links each RT structure and dose to its image series. The images and RT files do **not** need to live in the same folder.

```
from DicomRTTool.ReaderWriter import DicomReaderWriter

reader = DicomReaderWriter()
reader.walk_through_folders("/path/to/dicom")

# What ROIs exist across everything that was found?
all_rois = reader.return_rois(print_rois=True)
```



Step 2 — Survey: write a metadata manifest

Before committing to an export, get a one-row-per-series overview with `create_manifest`. With no ROIs selected yet it records **every** ROI it found, so you can see what is available and how large each structure is:

```
reader.create_manifest("/path/to/manifest.csv")
```



DicomRTTool

(Python)

pip install · GitHub

★ Recommended — grab the Release



THE STARTING POINT

Sifting through the data: The Results

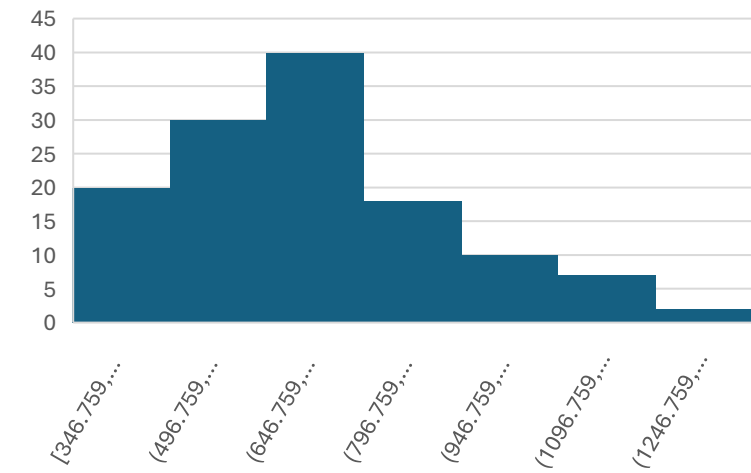
Installation

```
pip install DicomRTTool
```



	A	B	C	D	E	F	G	H	I	J	K	L
1	patient_has	study_has	series_has	spacing_x	spacing_y	spacing_z	esophagu	heart cc	lung-left c	lung-right	lungs-tota	spinal-cord cc
2	P204b9f9c	ST4d2cc00	SEf653e65	0.976563	0.976563	3	45.816				2898.972	54.854
3	Pb03f2db8	ST5048be6	SE167acc7	0.976563	0.976563	3	21.727	346.759			2850.208	44.735
4	P60620617	STe599c89	SE206f6d3	0.976563	0.976563	3	48.157					74.015
5	P7ef58c41	ST13b227k	SE4748df8	0.976563	0.976563	3	28.241		1342.323	1532.387		51.916
6	P9d17c899	ST37ada99	SEdbf6517	0.976563	0.976563	3	56.268	680.832			6281.096	121.811
7	P8732e2e4	STa072ff0a	SE35975df	0.976563	0.976563	3	59.718		1898.901	1673.701		68.773
8	Pb6e69e8f	STb17f27b	SE26093d7	0.976563	0.976563	3	45.264				3754.094	82.678
9	Pa9ec4ab1	STb6a5f97	SE1abe802	0.976563	0.976563	3	40.555		1623.874	2231.217		87.344
10	Pad426dd	ST2fab0c	SE12ba285	0.976563	0.976563	3			1752.133	1981.868		67.308
11	P02be2982	ST732c84a	SE85f1537	0.976563	0.976563	3	66.545		1836.894	2329.207		68.805
12	P5b5a223c	STb0d9038	SE2d88af4	0.976563	0.976563	3	31.274			2820.763		54.448
13	P45d43669	STd440db9	SE24d4680	0.976562	0.976562	3	42.017		2592.167	3332.47		94.222
14	P8c3b936c	ST2d498b4	SEc3e669c	0.976563	0.976563	3	52.116	844.059			5260.36	71.42

Volume [cc]



```
reader.create_manifest("/path/to/manifest.csv")
```



READ THE MANIFEST

Spotting the outliers



One row per series: spacing and per-ROI volume in cc. Sort any column and the bad cases fall to the ends.

COLUMN · spacing_z

Geometry outliers

A 5 mm slice in a 3 mm cohort. Odd thickness distorts every downstream resample — flag it before converting.

COLUMN · lung cc

Volume outliers

A lung at 200 cc where the cohort sits near 2000 cc. Almost always a partial or mis-drawn contour.

BLANK CELL

Missing structures

An empty cc cell means that ROI isn't on the series. Decide up front: exclude it or map an alias.

The spacing and volume histograms from the manifest turn the whole cohort into one glance — outliers live in the tails.

MATCH THE NAMES

When 'empty' means 'misnamed'

A blank **cc** column often isn't a missing organ — it's the same ROI stored under a different name.



IN THE RTSTRUCT

Left Lung · Lung-Left · L Lung · lung_l

one organ, four spellings



ONE CANONICAL NAME

lung_l

now its **cc** column fills in

one canonical name, many aliases

```
reader.set_contour_names_and_associations(  
    contour_names=["lung_l", "lung_r", "cord"],  
    associations=[ROIAssociationClass("lung_l", ["left lung", "lung-left", "l lung"])],  
)  
# aliases match case-insensitively
```

github.com/brianmanderson/Dicom_RT_and_Images_to_Mask · Step 3 — Select: choose ROIs & map aliases

Choosing an output voxel size



The survey shows native spacing varies. Pick **one** target so every case lands on the same grid.

THE DECISION

Isotropic 1×1×1 mm

Uniform geometry, model-friendly, but larger arrays.

Preserve native (0.975×0.975×3)

Smaller, but mixed geometry across the cohort.

```
resample on export
```

```
reader.write_to_folder(  
    "out",  
    output_spacing=(1.0, 1.0, 3.0),  
)
```

```
# or resample any handle directly:  
resample_to_spacing(img, (1, 1, 3), "Linear")
```

Image Linear interpolation

Dose Linear, onto resampled image grid

Mask Nearest-neighbor

SET THE TARGET GRID

Choosing an output voxel size



The survey shows native spacing varies. Pick **one** target so every case lands on the same grid.

THE DECISION

Isotropic 1×1×1 mm

Uniform geometry, model-friendly, but larger arrays.

Preserve native (0.975×0.975×3)

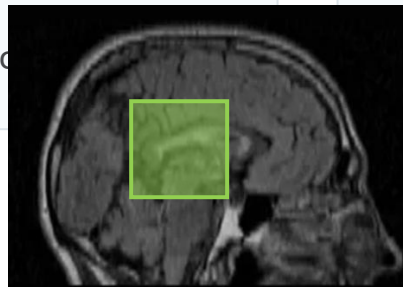
Smaller, but mixed geometry across the cohort.

```
resample on export
```

```
reader.write_to_folder(  
    "out",  
    output_spacing=(1.0, 1.0, 3.0),  
)
```

```
# or resample any handle directly:  
resample_to_spacing(img, (1, 1, 3), "Linear")
```

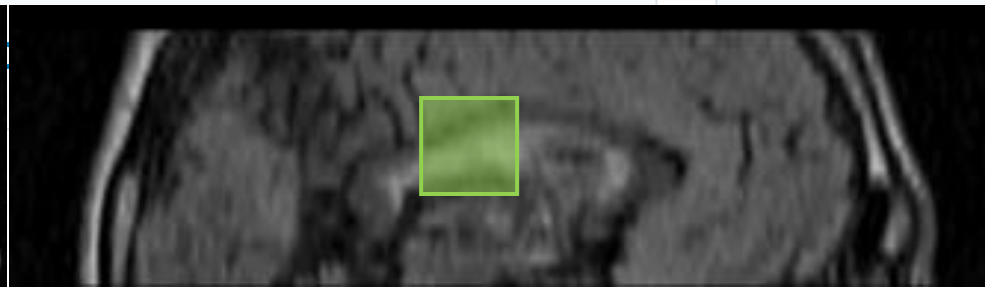
Image Linear interpolation



3mm slice thickness

NumPy array (60, 512, 512)

Nearest-neighbor



1mm slice thickness

NumPy array (60, 512, 512)

Choosing an output voxel size



The survey shows native spacing varies. Pick **one** target so every case lands on the same grid.

THE DECISION

Isotropic 1×1×1 mm

Uniform geometry, model-friendly, but larger arrays.

Preserve native (0.975×0.975×3)

Smaller, but mixed geometry across the cohort.

```
resample on export
```

```
reader.write_to_folder(  
    "out",  
    output_spacing=(1.0, 1.0, 3.0),  
)  
  
# or resample any handle directly:  
resample_to_spacing(img, (1, 1, 3), "Linear")
```

Image Linear interpolation

Dose Linear, onto resampled image grid

Mask Nearest-neighbor

Preserving clinical metadata



Ask for the DICOM tags you want — they land in a grouped, versioned `metadata.json` (schema v2), under `image.tags`.

request extra DICOM tags

```
reader = DicomReaderWriter(  
    Contour_Names=["lung_1", "cord"],  
    image_sitk_string_keys={  
        "PatientAge": "0010|1010",  
        "Manufacturer": "0008|0070",  
    },  
)
```

metadata.json – grouped (schema v2), one per series

```
{  
  "schema_version": 2, "anonymized": true,  
  "case": {"patient": "P1a2b3c", ...},  
  "image": {  
    "modality": "CT",  
    "export": {"effective_spacing_mm": [1.0, 1.0, 3.0], "resampled": true},  
    "tags": {"PatientAge": "063Y", "Manufacturer": "SIEMENS"} ← yours  
  },  
  "structures": [{"rois": [{..., "volume_cc", "exported_file" ...}]}]  
}
```

github.com/brianmanderson/Dicom_RT_and_Images_to_Mask · [README](#) → Grouped metadata (schema v2)

THE PAYOFF

DICOM → NIfTI in one call

Voxel size, masks, dose, metadata, and the anon key — one call writes them all to a tidy per-case tree.



convert.py

```
from DicomRTTool.ReaderWriter import DicomReaderWriter
reader = DicomReaderWriter(
    Contour_Names=["lung_l", "lung_r", "cord"],
    image_sitk_string_keys={          # → image.tags
        "PatientAge": "0010|1010",
        "Manufacturer": "0008|0070"},
)
reader.walk_through_folders("/dicom")
reader.write_to_folder(
    "/out", output_spacing=(1.0, 1.0, 3.0),
    anonymize=True, salt="MyProjectSalt")
```

output tree

```
out/
  <patient>/<study>/<series>/
    image.nii.gz
    masks/
      lung_l.nii.gz  lung_r.nii.gz
      cord.nii.gz
    doses/plan.nii.gz
    metadata.json    # grouped (schema v2)
    manifest.csv
    anonymization_key.json
```

GROW THE DATASET

One ID, always the same hash



PatientID, **StudyInstanceUID**, and **SeriesInstanceUID** each map to a deterministic SHA-256 hash — same input + same salt = the same hash, every run.

PATIENT ID

1234567 → **P0c2ef36d55**

hash_patient()

STUDY UID

1.3.6...793 → **ST2e82a7fd7166**

hash_study()

SERIES UID

1.3.6...421 → **SE73dee5c80eef**

hash_series()

ADD MORE PATIENTS ANYTIME

New MRNs and UIDs hash into fresh case folders and pool into the same de-identified set — no manual ID assignment, no collisions.

RE-ADD ONE PATIENT LATER → IT SLOTS RIGHT BACK IN

The same MRN and UIDs regenerate the **identical** P... / ST... / SE... hashes — so every file lands in the same case folder and updates in place. Never a duplicate.

The reverse-lookup key (hash ↔ MRN) lives in **anonymization_key.json** — kept offline and access-controlled, it never travels with the dataset.

CHOOSE A FORMAT

NIfTI vs NumPy



CRITERION	NIfTI	NumPy (.npy / .npz)
Spatial metadata	Built-in affine & spacing	Manual / sidecar
Tooling	ITK, 3D Slicer, viewers	Python-only
Interoperability	Standard, portable	Convenient, ad-hoc
Best for	Images, dose, masks	Fast batch loading / tensors

Recommendation: NIfTI as the interoperable source of truth; NumPy for training-time speed.

The whole picture

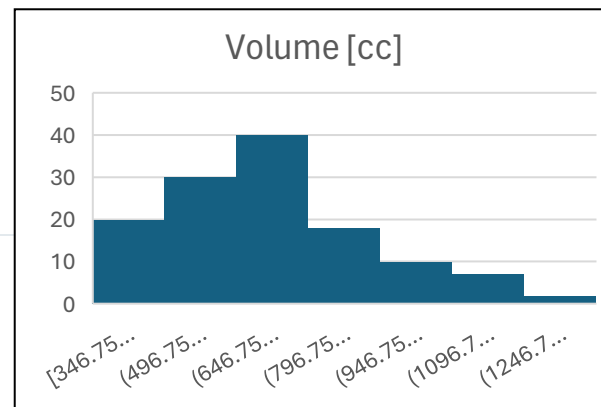
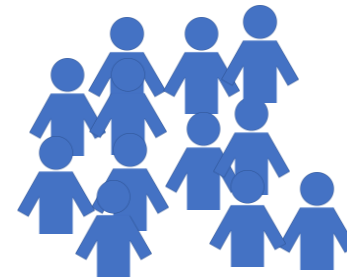
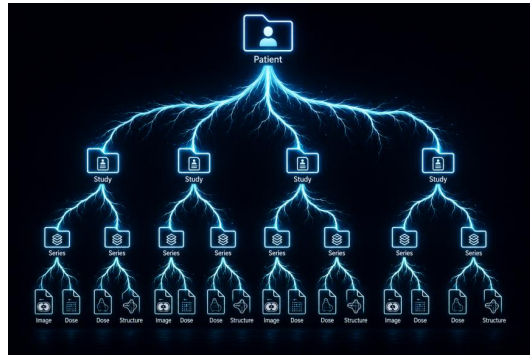
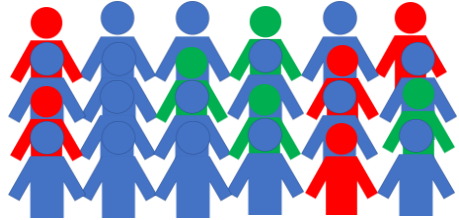
Everything in one place



PATIENT ID

1234567 → **P0c2ef36d55**

hash_patient()



THE THREE-SESSION ARC

From pixels to patients

Session 1 · You are here

From DICOM to a generalized dataset

Session 2

Building reproducible pipelines in PyTorch

Session 3

Integrating & monitoring models in the clinic



Thank you

Brian Mark Anderson, PhD · Radiation Medicine & Applied Sciences

UC San Diego Health

RETHINKING MEDICAL PHYSICS